

AI usage for OSN-MP-1 mid submission

1) Links of AI-chats

- i) <https://claude.ai/share/454d2d9b-716a-4fab-9ebd-422f1eafc154>
- ii) <https://claude.ai/share/3fb79fc0-a64c-46e1-99c5-f89332d282e6>
- iii) <https://claude.ai/share/aadde535-8775-4d35-bbb9-56c09360372b>
- iv) <https://claude.ai/share/9d993a37-76d1-4ce6-abe3-c3f68a0cb2bb>
- v) <https://claude.ai/share/e10c2a96-ee52-45f5-92d1-4925db078f78>
- vi) <https://share.gemini.google/GD8pIzDHQB2n>
- vii) <https://share.gemini.google/pBhftE0FUgBZ>

2) Pics [were with other random prompts]

ok so how to approach A now (A1 and A2)\

To finish A1 and A2, you need to replace the fake prompt in our skeleton with the actual system data.

There is a massive 'gotcha' in the assignment instructions for A1 that catches a lot of people: "The directory the shell is started in becomes its home directory." This means your shell's ~ is not /home/soubhagya. It is whatever folder you are in when you run ./shell.out.

Here is how to approach building the actual prompt and wiring it into your loop.

1. Update shell.h

Include required headers

You need a few more POSIX libraries to get the username and user ID. Add these to your include/shell.h file:

```
c
#include <sys/types.h>
#include <pwd.h>
```

2. Capture the Shell's Home Directory

Store it right when the program starts

In your main.c, you need to figure out where the shell was launched from before the infinite loop starts, so you have it as a reference for the ~ symbol later.

```
c
int main() {
    char home_dir[MAX_PATH_SIZE];
    if (getcwd(home_dir, sizeof(home_dir)) == NULL) {
        perror("getcwd error");
        return 1;
    }

    char input[MAX_INPUT_SIZE];
    // ... (while loop starts here)
}
```

3. Write the Prompt Logic (Part A1)

Pass the home_dir to your function

Here is how to get the username, hostname, and handle the path replacement. Replace your print_prompt function with this logic:

```
c
void print_prompt(const char *home_dir) {
    char cwd[MAX_PATH_SIZE];
    char hostname[256];

    // 1. Get Username (POSIX standard way)
    struct passwd *pw = getpwuid(getuid());
    const char *username = pw ? pw->pw_name : "unknown";

    // 2. Get Hostname
    if (gethostname(hostname, sizeof(hostname)) != 0) {
        strcpy(hostname, "unknown");
    }

    // 3. Get Current Working Directory
    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd error");
        return;
    }

    // 4. The '~' logic: check if cwd starts with home_dir
```

```
c
// 4. The '~' logic: check if cwd starts with home_dir
size_t home_len = strlen(home_dir);

// Ensure it's a direct match and stops at a folder boundary (avoids false p
if (strncmp(cwd, home_dir, home_len) == 0 && (cwd[home_len] == '/' || cwd[home_len] == '\0')) {
    // Print with '~' replacing the home_dir part
    printf("%s@%s:~%s ", username, hostname, cwd + home_len);
} else {
    // Outside of home_dir, print absolute path
    printf("%s@%s:%s ", username, hostname, cwd);
}

fflush(stdout);
}
```

4. Finalize the REPL (Part A2)

Wire it all together

Now, just update your while(1) loop inside main() to pass the home_dir into the prompt, and keep the fgets code we wrote earlier to consume input.

```
c
while (1) {
    print_prompt(home_dir);

    if (fgets(input, MAX_INPUT_SIZE, stdin) == NULL) {
        printf("\n");
        break; // Exit on Ctrl+D
    }

    input[strcspn(input, "\n")] = 0; // Strip newline

    if (strlen(input) == 0) {
        continue; // Show prompt again on empty enter
    }

    printf("Executed: %s\n", input);
}
```

Once you copy this in, run make and ./shell.out. You should see your actual username and hostname, and the path should show ~! If you're confident it works, commit it:

```
Bash
git add .
git commit -m "feat: completed A1 shell prompt and A2 user input"
git push
```

Ready to start Lexing and Parsing?

Yes

so how do you start writing the code like give hints what to add instead of full maybe a pseudo code

Here is a blueprint using pseudo-code and structural hints. This will help you write the actual C code without violating your request to figure it out yourself.

To avoid being penalized for monolithic code, split this into three distinct files.

1. Step 1: Set up your global definitions

include/shell.h

This file will hold your constants and necessary standard library includes.

```
c
// Include guards (e.g., #ifndef SHELL_H ...)
// Include libraries: stdio, stdlib,unistd, string, pwd, sys/types

// Define your constraints based on the assignment
#define MAX_INPUT 1024
#define MAX_PATH 4096

// Declare the prompt function so main can use it
void print_prompt(const char *shell_home);
```

2. Step 2: Write the prompt logic

src/prompt.c

This file handles fetching system info and doing the string math for the ~ replacement.

```
c
// Include shell.h

void print_prompt(const char *shell_home) {
    // 1. Declare buffers for cwd and hostname

    // 2. Fetch Username
    // hint: struct passwd *p = getpwuid(getuid());
    // string username = p->pw_name;

    // 3. Fetch Hostname
    // hint: gethostname(hostname_buffer, size);

    // 4. Fetch Current Working Directory (CWD)
    // hint: getcwd(cwd_buffer, size);

    // 5. The Path Logic (The tricky part)
    // Check if cwd_buffer starts with shell_home.
    // hint: use strncmp(cwd_buffer, shell_home, strlen(shell_home))

    // If it starts with shell_home AND the next character in cwd_buffer is eit
    // hint: printf("%s@%s:~%s ", username, hostname, cwd_buffer + strlen(
    // Else:
    // Print the prompt normally with the full absolute path
    // hint: printf("%s@%s:%s ", username, hostname, cwd_buffer);

    // 6. Force the prompt to display immediately
    // hint: fflush(stdout);
}
```

The Fix

Open your `src/hop.c` file and update both functions to use `.frequency` consistently, and swap `strstr()` for `strcasestr()` so lowercase queries match uppercase directory names smoothly.